

1. JVM, JDK, JRE, JAR, class loading, build tools

1. В чем разница между JDK, JRE и JVM?
2. Какие части входят в JDK и какие из них реально используются во время разработки, сборки и запуска приложения?
3. Что происходит с Java-кодом от .java файла до выполнения инструкций на процессоре?
4. Что такое байткод и что хранится в .class файле?
5. Как работает механизм class loading в Java?
6. Какие class loader'ы существуют в стандартной JVM и за что отвечает каждый из них?
7. Чем отличаются loading, linking и initialization класса?
8. В какие моменты класс действительно инициализируется?
9. В каком порядке выполняются static-поля, static-блоки и инициализация родительских классов?
10. Что такое JAR-файл и чем обычный JAR отличается от executable JAR?
11. Какую роль играют MANIFEST.MF, Main-Class и classpath при запуске приложения?
12. Чем classpath отличается от module path, зачем появился JPMS и какие проблемы классического classpath он решает?
13. Что такое strong encapsulation модулей и как она влияет на reflection?
14. Зачем нужны системы сборки вроде Maven и Gradle, если можно компилировать код напрямую через javac?
15. Как в общих чертах работают системы сборки и что они делают помимо компиляции?
16. Как build tool разрешает транзитивные зависимости и конфликты версий?
17. Что такое annotation processing и на каком этапе сборки он работает?
18. Что делает JIT-компилятор и зачем он нужен, если код уже скомпилирован в байткод?
19. Как в общих чертах работают profiling, tiered compilation, inlining, devirtualization и escape analysis?

20. Что такое JMH и почему микробенчмарки “на коленке” в Java часто дают ложные выводы о производительности?

2. Типы данных, объектная модель, память

21. Чем примитивные типы отличаются от ссылочных, почему примитивы не являются полноценными объектами и какие различия между ними реально важны в прод-коде?

22. Какие размеры и диапазоны имеют примитивные числовые типы и какие из этих различий реально важны на практике?

23. Как работает переполнение целых чисел в Java и почему оно не вызывает исключение?

24. Как работают float и double, почему $0.1 + 0.2 \neq 0.3$, что такое NaN, Infinity и -0.0, и как все это ведет себя в сравнениях?

25. Что такое boxing и unboxing и какие проблемы они создают?

26. Что такое Integer Cache и почему Integer иногда сравнивается через == “как будто правильно”?

27. Почему null особенно опасен для оберток при авто-распаковке?

28. Какие методы класса Object являются фундаментальными для Java-модели объектов и что важно знать про корректное переопределение equals(), hashCode() и toString()?

29. Чем отличается идентичность объекта от логического равенства?

30. Почему String immutable, как работают String Pool и строковые литералы, что такое immutable-объект в целом и почему иммутабельность особенно важна в многопоточном коде?

31. Что такое defensive copying и в каких случаях он нужен?

32. Что происходит в памяти при создании объекта в Java?

33. Какие области памяти есть в JVM и какие из них общие для всех потоков, а какие принадлежат потоку?

34. Что хранится в heap, stack, metaspace, code cache и native memory?

35. Что такое stack frame и что в нем находится?

36. Где живут локальные переменные, параметры метода и ссылки на объекты?

37. Что именно находится в heap: сами объекты, массивы, поля объектов или только ссылки?
38. Почему в Java все передается по значению, даже когда речь идет об объектах, и что означает фраза "ссылка на объект передается по значению"?
39. Что такое strong, soft, weak и phantom references и где они реально применяются?
40. Что такое GC roots и как связаны достижимость объекта, его жизненный цикл и возможность его удаления сборщиком мусора?

3. Generics, wildcard'ы, PECS

41. Зачем в Java вообще нужны generics и какую проблему они решают?
42. Почему generics в Java инвариантны, чем это отличается от ковариантности массивов и почему `List<Object>` и `List<String>` — это несовместимые типы?
43. Что такое type erasure, почему generics в Java реализованы именно так и какие последствия это имеет для runtime-поведения программы?
44. Что такое bridge methods и зачем компилятор их генерирует?
45. Почему внутри generic-кода нельзя создать `new T()` и почему нельзя напрямую создавать массивы generic-типов?
46. Что такое raw type и почему его использование считается опасным?
47. Что такое bounded type parameter, когда нужен `T extends ...` и как работают множественные upper bounds у generic-параметров?
48. Как работают wildcard'ы в Java generics: чем `?` отличается от именованного generic-параметра, когда использовать wildcard, а когда generic-метод, что означают `? extends T` и `? super T`, в чем состоит принцип PECS, чем отличаются `List<?>`, `List<? extends Object>` и `List<Object>`, и зачем нужен wildcard capture?
49. Какие ограничения и подводные камни generics в Java сильнее всего влияют на дизайн API и производительность?
50. Как спроектировать generic API так, чтобы он был гибким, но не превращался в нечитаемую абстракцию?

4. Исключения, Optional, records, sealed classes

- 51. Как устроена иерархия Throwable в Java?
- 52. Чем Error, Exception и RuntimeException отличаются концептуально?
- 53. В чем практическая разница между checked и unchecked исключениями и в каких случаях checked exceptions действительно полезны, а в каких мешают?
- 54. Какие правила действуют для переопределения методов с checked exceptions?
- 55. Чем throw отличается от throws?
- 56. Каков порядок выполнения try, catch и finally, включая случаи с return?
- 57. Какие подводные камни есть у finally?
- 58. Как работает try-with-resources, что такое suppressed exceptions и чем AutoCloseable отличается от Closeable?
- 59. Когда исключение нужно заворачивать в другое, а когда лучше пробрасывать как есть?
- 60. Как правильно проектировать кастомную иерархию исключений для домена и инфраструктуры?
- 61. Почему “поймать, залогировать и выбросить дальше” часто является плохой практикой?
- 62. Когда Optional уместен как возвращаемое значение, а когда его лучше не использовать, в том числе в полях сущности, параметрах методов и элементах коллекций?
- 63. Чем отличаются map, flatMap, orElse, orElseGet и orElseThrow у Optional?
- 64. Что такое records и sealed classes, чем они отличаются от обычных классов, какие ограничения у них есть и в каких случаях они реально упрощают модель?

5. Коллекции

- 65. Как устроена иерархия коллекций в Java и почему Map формально не наследуется от Collection?
- 66. Чем массив принципиально отличается от коллекции и когда лучше использовать массив, а когда коллекцию?

67. Что такое immutable collection и чем она отличается от unmodifiable view?
68. Почему для корректной работы коллекций критически важен контракт equals() и hashCode()?
69. Что такое fail-fast iterator и почему возникает ConcurrentModificationException?
70. Как устроен ArrayList внутри, чем отличаются size и capacity и как растет его внутренний массив?
71. Какова асимптотическая и реальная стоимость основных операций в Java-коллекциях и от чего она зависит?
72. Почему ArrayList на практике часто быстрее LinkedList, даже когда асимптотика у некоторых операций выглядит похоже?
73. Как устроен LinkedList под капотом, почему доступ по индексу в нем дорогой, почему он обычно проигрывает по памяти и cache locality и в каких редких случаях его использование все еще оправдано?
74. Как работает HashMap под капотом: вычисление бакета по hashCode, размазывание хэша, степень двойки у таблицы, put, get, коллизии, переход списка в дерево и обратно, load factor, threshold, resize, работа с null, влияние плохого hashCode и последствия нарушения контракта equals() и hashCode()?
75. Почему HashSet фактически построен поверх HashMap и в чем между ними концептуальная разница?
76. Когда выбирать LinkedHashMap, чем полезен access-order и как на нем строят LRU-cache?
77. Как устроен TreeMap, чем он отличается от HashMap и какие риски появляются, если Comparator не согласован с equals()?
78. Чем TreeSet отличается от HashSet и когда цена сортировки оправдана?
79. Как устроен ConcurrentHashMap в JDK 8+ и чем он отличается от старых сегментированных реализаций?
80. Почему CopyOnWriteArrayList полезен в одних сценариях и катастрофически плох в других?
81. Когда лучше выбрать ArrayDeque, чем Stack или LinkedList?
82. Как устроен PriorityQueue и в каких задачах он полезен?

- 83. Какие особенности и сценарии применения у `LinkedHashSet`, `EnumSet`, `EnumMap`, `WeakHashMap`, `IdentityHashMap`, `ConcurrentSkipListMap`, `BlockingQueue`, `DelayQueue` и `SynchronousQueue`?
- 84. Что произойдет, если использовать изменяемый ключ в `HashMap` или элемент в `HashSet`?
- 85. Чем `Iterator` отличается от `ListIterator` и когда второй действительно нужен?
- 86. Почему legacy-коллекции вроде `Vector` и `Stack` обычно считаются плохим выбором в современном коде?
- 87. Как работает `subList()` и какие подводные камни у него есть?
- 88. Как работают обертки вроде `Collections.synchronizedList()` и в чем их ограничения?
- 89. Чем `List.of()`, `Set.of()` и `Map.of()` отличаются от старых способов создания неизменяемых коллекций?
- 90. Чем отличаются коллекции с точки зрения порядка элементов: `insertion order`, `access order`, `natural order` и отсутствия порядка?
- 91. Чем отличаются `ArrayList`, `LinkedList`, `HashMap` и `TreeMap` по накладным расходам на память и почему это важно?
- 92. Как выбирать коллекцию под конкретную нагрузку с учетом чтения, записи, поиска, порядка, памяти и конкуренции?

6. Lambdas, functional style, Stream API

- 93. Что такое функциональный интерфейс и зачем он нужен в Java?
- 94. Чем `lambda`-выражение отличается от анонимного класса?
- 95. Что такое `method reference` и когда он действительно улучшает код, а когда только ухудшает читаемость?
- 96. Почему локальные переменные, захватываемые `lambda`-выражением, должны быть `effectively final`?
- 97. Что такое замыкание в контексте Java `lambda`-выражений и как оно работает?
- 98. Что такое `Stream` в Java и чем он концептуально отличается от коллекции?
- 99. Что такое `stream pipeline` и почему промежуточные операции ленивые?
- 100. Какие операции у `Stream` являются `intermediate`, а какие `terminal`?

- 101. Что такое stateless и stateful intermediate operations?
- 102. Почему sorted() и distinct() могут резко менять стоимость pipeline?
- 103. Чем map() отличается от flatMap()?
- 104. Как Stream работает под капотом на уровне single-pass обработки элементов и почему один и тот же Stream нельзя использовать повторно?
- 105. Почему reduce() и collect() — это не одно и то же?
- 106. Что такое mutable reduction и из чего состоит Collector: supplier, accumulator, combiner, finisher?
- 107. Что означают характеристики collector'а вроде CONCURRENT, UNORDERED и IDENTITY_FINISH?
- 108. Как работают groupingBy(), partitioningBy() и downstream collectors?
- 109. Почему побочные эффекты внутри stream-операций считаются опасной практикой?
- 110. Как работают short-circuit операции вроде findFirst(), findAny(), anyMatch() и allMatch(), когда parallelStream() действительно может помочь, а когда только ухудшит ситуацию, и какую роль здесь играет Spliterator?

7. Процессы, потоки, JMM, синхронизация, executors, atomics, futures

- 111. Чем процесс отличается от потока на уровне ОС и как это отражается в Java-приложении?
- 112. Чем platform thread отличается от virtual thread, где virtual threads реально полезны и где они не решают фундаментальную проблему?
- 113. Что такое Java Memory Model и какую проблему она решает?
- 114. Чем отличаются atomicity, visibility и ordering?
- 115. Что такое happens-before и почему без него многопоточный код нельзя считать корректным?
- 116. Что такое race condition и чем она отличается от data race?
- 117. Как работает synchronized на концептуальном уровне?
- 118. Что такое monitor enter и monitor exit?
- 119. Какие гарантии дает volatile, а какие не дает?

- 120. Почему `volatile` не делает составные операции атомарными?
- 121. Какие особые гарантии публикации дает `final` для полей объекта?
- 122. Когда использовать `synchronized`, а когда `ReentrantLock`?
- 123. В чем смысл `ReadWriteLock` и `StampedLock`, где они полезны и какие у них есть риски?
- 124. Почему `wait()`, `notify()` и `notifyAll()` считаются низкоуровневым инструментом и чем их обычно заменяют?
- 125. Когда использовать `CountDownLatch`, `CyclicBarrier`, `Semaphore` и `Phaser`?
- 126. Что такое `ThreadLocal`, где он полезен и чем опасен в `thread pool`'ах?
- 127. Чем `Executor`, `ExecutorService` и `ScheduledExecutorService` отличаются друг от друга?
- 128. Как подбирать размер пула потоков для CPU-bound и I/O-bound задач?
- 129. Чем `newFixedThreadPool()`, `newCachedThreadPool()` и `newWorkStealingPool()` отличаются по модели работы?
- 130. Что такое `ForkJoinPool`, как работает `work stealing`, зачем существует `common pool` и какие риски есть у его бездумного использования?
- 131. Чем `Future` ограничен как абстракция асинхронного результата?
- 132. Чем `CompletableFuture` принципиально мощнее обычного `Future`?
- 133. Чем отличаются `thenApply()`, `thenCompose()`, `thenCombine()` и `allOf()`, и почему выбор `executor`'а для `CompletableFuture` принципиально важен?
- 134. Как в Java правильно организуется отмена задач и что такое `cooperative interruption`?
- 135. Что происходит при `Thread.interrupt()`, почему это не "жесткое убийство" потока и чем `deadlock`, `livelock` и `starvation` отличаются друг от друга?
- 136. Как работают атомики под капотом: что такое `CAS`, чем `AtomicLong` отличается от `LongAdder`, что такое ABA-проблема и `false sharing`?

8. GC, профилирование, диагностика

- 137. Почему современные JVM используют анализ достижимости от GC roots, а не простой `reference counting`?
- 138. Что такое `stop-the-world` паузы и почему большинство GC опираются на `generational hypothesis`?

139. Как в общих чертах устроены young collection, old collection, promotion и aging объектов?
140. Чем Serial GC, Parallel GC, G1 GC, ZGC и Shenandoah отличаются по целям, компромиссам и типичным сценариям применения?
141. Как работает G1 на уровне регионов и почему его часто выбирают по умолчанию?
142. В чем идея low-latency collectors вроде ZGC и Shenandoah и какие у них есть ограничения?
143. Что считается “утечкой памяти” в Java, если память чистит GC, и какие типовые причины утечек встречаются чаще всего?
144. Какие разновидности OutOfMemoryError бывают и почему heap, metaspace, direct memory и native thread exhaustion — это не одно и то же?
145. Чем sampling profiler отличается от instrumentation profiler?
146. Какие способы профилирования и диагностики JVM должен знать senior-разработчик?
147. Когда использовать jstack, jcmd, JFR, JMC, VisualVM, heap dump и thread dump, и что каждый из этих инструментов обычно помогает понять?
148. Как читать GC-логи и какие метрики GC особенно важны при разборе проблем производительности?
149. Как анализировать heap dump, чтобы находить утечки памяти, крупные ретейнеры и подозрительные графы объектов?
150. Как отличить CPU-bound проблему от блокировок, contention, I/O wait и утечки памяти и как выстроить нормальный performance investigation workflow от симптома к подтвержденной причине?